P1 LISTA

1-Defina o modelo rasp e qual tipos de parâmetros nele permitido:
Variação do modelo ram em que o programa pode ser modificado em tempo de execução.
Os operando permitidos no modelo são: =L(valor i) e (inteiro não negativo) registradores

Consiste de :
- uma fita somente de entrada
-uma somente de saída
-**um programa armazenado na memoria**
-uma memória do tipo acumulador.
-As fitas são Divididas em quadros onde cabe um inteiro;

-A memória consiste em uma seuqencia de registradores ,r0,r1..rn**, onde em cada um cabe um inteiro e não há limite para a memória.**

3-O que é uma máquina de turing k fitas?

É uma septupla onde:
(Q, T, I, d, b, q0 , qf), onde:

-Q é o conjunto de estados.

-T é o conjunto de símbolos de fita.

-I é o conjunto de símbolos de entrada; IE T

- b esta em T-I, é o espaço em branco.

- qo é o estado inicial.

- qf é o estado final

- g função de próximo-movimento.

4-Uma indução matemática resulta em que tipo de resultado? O processo de indução matemática resulta em algoritimos recursivos.

5-Como diferenciar modelo ram e modelo rasp?

MRASP:

- O programa ocupa espaço na memória, fica armazenado em registadores

-É possível modificar o programa em tempo de execução

-instruções de endereçamento indireto não são permitidas

MRAM:

-O programa não é armazenado em registradores, e sim executado diretamente com base em instruções fornecidas de forma sequencial e imediata.

-As instruções são fixas e executadas conforme definidas inicialmente

-Instruções de endereçamento indireto sã permitidas.

6-O que é complexidade de tempo: O tempo necessário por um algoritmo expresso como uma função do tamanho do problema

7-O que é complexidade assintótica de tempo: O comportamento limítrofe da complexidade conforme o tamanho aumenta

8-O que é um modelo de computação: é a definição de um conjunto de operações que podem ser usadas numa computação e seus respectivos custos.

● Somente assumindo certo modelo de computação é possível analisar os recursos computacionais requeridos, como tempo de execução e espaço de armazenamento, ou discutir as limitações de algoritmos ou computadores

9-Os principais modelos de computação são : – Máquina de Acesso Aleatório (RAM) – Máquina de Acesso Aleatório com Programa Armazenado , Máquina de Turing.

10-Como definir a complexidade de tempo e espaço no modelo ram? :

Serão utilizados dois critérios de custos:

– Critério de Custo Uniforme; e

– Critério de Custo Logarítmico.

11-Como é definida Complexidade logaritima de espaço: o é definida sobre a soma de todos os registradores, incluindo o acumulador, de l(xi),

12-Qual a definição de uma máquina de turing? : consiste em uma fita infinita dividida em células, uma cabeça de leitura/escrita que se move sobre a fita e um conjunto finito de estados, juntamente com um conjunto de regras de transição. Essas regras determinam como a máquina lê, escreve e move a cabeça pela fita, permitindo que ela simule qualquer algoritmo computável.

13-O que é uma relação de recorrência?

é uma maneira de definir uma função através de uma expressão envolvendo a mesma função. Relações de Recorrência Exemplo: A sequência de Fibonacci.

F(n) = F(n-1) + F(n-2) F(2) = 1 F(1) = 1

14-Quais as Principais Técnicas usadas para solucionar Recorrencias:

Palpites Inteligentes

- Método de Substituição
- Relações de Divisão e Conquista
- Recorrências com histórico completo

15-*O QUE FAZ O PROGRAMA ABAIXO? E qual modelo ele esta?

-Modelo rasp,instrução store *R1 indica endereçamento indireto.

-Le um numero indeterminado de números inteiros maiores que zero para um vetor a partir da posição 100 da memória.

LISTA P2

1-Como provamos a corretude de um Algoritimo?

Para provar que um algoritmo é correto usamos o Laço Invariante ou Loop Invariante.

Uma invariante de loop é uma condição que é verdadeira antes e depois de cada iteração do loop. E provamos os seguintes pontos.

2Quais os passos para provar um invariante?

É necessário 3 pontos para provar um invariante.

Inicialização: O invariante deve ser verdadeiro antes da primeira iteração do laço.

Manutenção: O invariante, se é verdadeiro antes de uma iteração i, permanecerá verdadeiro antes da iteração i+1.

Término: Ao fim da execução do loop o invariante implica a corretude do algoritimo.

3- O que é invariante?

é uma característica (ou condição) que ocorre em um laço que é verdadeira imediatamente antes (e depois) de cada iteração do laço.

Busca Linear -> antes da t-ésima iteração temos que A[1...i-1] não contém um elemento com chave K.

Busca Binária-> antes da t-ésima iteração temos que esq<= dir e A[1...esq-1] e A[dir+1...n] não contém um elemento com chave K.

BubbleSort-> antes da t-ésima iteração temos que os últimos T elementos do vetor são os maiores e estão em ordem crescente.

SelectionSort-> antes da t-ésima iteração temos que o subvetor A[i..n-1] a variável “maior” , possui o índice do maior elemento do subvetor B[i...j-1]

MergeSort-> antes da t-ésima iteração temos que a lista resultante em A[Baixo...k-1] é ordenado e contém os menores elementos dos vetores]

InsertiorSort Interno-> antes da t-ésima iteração temos que os elementos A[j+1] até A[i-1] são maiores que a chave e foram movidos uma posição a frente.

InsertionSort Externo -> antes da t-ésima iteração temos que os primeiros i-1 elementos do vetor A estão ordenados.

Quicksort Interno -> antes da t-ésima iteração temos que todos elementos em A[baixo..i] são menores ou iguais ao pivô e os elementos em A[baixo..i] são menores ou iguais ao pivô e os elementos em A[i+1...j-1] são maiores que o pivô.

QuickSort Externo-> antes da t-ésima iteração temos que todos os elementos a esq do pivô são menores ou iguais ao pivô e os da direita >=.

LISTA P3(DINAMICA E GULOSA)

1-Diferencie Programação Gulosa de Programação Dinâmica

Ambos são aplicados em problemas de otimização e se diferenciam por:

-PD: Avalia todos os subproblemas possíveis para determinar a solução ótima.

-Sempre encontra uma solução ótima

-GULOSA:

-Baseia-se no ótimo local,escolha gulosa dentre os possíveis ,para determinar uma solução ótima global.

-Nem sempre encontra uma solução ótima, ou seja , não é aplicável em todos os problemas

2- Diferencie Programação Dinâmica e Divisão e Conquista

Tanto Programação Dinâmica quanto divisão e Conquista dividem o problema em subproblemas para determinar uma solução e se diferenciam por:

PD: =

-É aplicável em problemas de otimização

-Seus problemas de sobrepõem

DEC: =

-É aplicável em vários tipos de problemas

-seus problemas e seus problemas subproblemas não se sobrepoem

3-O que é programação dinâmica?

Programação dinâmica é uma técnica de otimização que resolve problemas ao dividir o problema principal em subproblemas menores que se repetem e compartilhando as soluções desses subproblemas para evitar trabalho repetido. Ao armazenar as soluções intermediárias, economiza-se tempo de computação.

4- Quando Utilizar programação dinâmica?

-O problema tem subproblemas sobrepostos que podem ser reutilizados.

-O problema tem uma subestrutura ótima

Problemas que envolvem otimização.

5-Explique os Passos principais para construir um Algoritimo de Programação Dinâmica?

-Caracterização da Subestrutura Ótima: Define-se como o problema maior pode ser resolvido combinando soluções ótimas de subproblemas menores.

-Definir a Recorrência: Usar uma formulação recursiva que relacione a solução do problema com as soluções dos subproblemas.

-Computar Soluções de Maneira Ascendente: Resolver os subproblemas menores primeiro, armazenando suas soluções, e depois usar essas soluções para resolver subproblemas maiores.

-Reconstruir a Solução Ótima: Caso seja necessário, pode-se rastrear as decisões feitas para construir uma solução explícita, além do valor ótimo.

6- Como Identificar uma Subestrutura Ótima? E Quais são suas Caracteristicas?

Deve verificar se o problema pode ser decomposto em **subproblemas menores** cujas soluções ótimas contribuem para a solução do problema maior.

As suas características são: Divisão de Subproblemas;Recursividade;Reutilização de soluções.

7-O que é Programação Gulosa e como Funciona?

Os algoritmos gulosos resolvem problemas de otimização fazendo a escolha localmente ótima em cada passo, na esperança de que essas escolhas resultem em uma solução globalmente ótima. A diferença em relação à programação dinâmica é que o algoritmo guloso toma decisões com base apenas no momento atual, sem considerar subproblemas futuros ou passados.

8- Quais os passos para resolver por programação Gulosa?

-Escolha gulosa: Em cada passo, faz-se a escolha que parece melhor naquele momento.

-Prova da propriedade de escolha gulosa: Mostra-se que sempre existe uma solução ótima que inclui a escolha gulosa feita em cada etapa.

-Prova da subestrutura ótima: Após fazer a escolha gulosa, o problema restante deve ser solucionável de maneira ótima com o mesmo procedimento**.**

9- Quando utilizar programação Gulosa?

-Problemas com a propriedade de escolha gulosa: Se há garantia de que a escolha localmente ótima leva à solução globalmente ótima.

-Subestrutura ótima: A solução do problema pode ser decomposta em subproblemas de maneira semelhante à programação dinâmica, mas sem a necessidade de recalcular ou armazenar subproblemas.

10-Explique o Método Divisão e conquista para construção de algoritmos?

O método de Divisão e Conquista é uma técnica de construção de algoritmos que resolve problemas complexos dividindo-os em subproblemas menores, resolvendo cada um de forma independente e combinando suas soluções para formar a solução do problema original.

P4 LISTA (PROBLEMAS POLINOMIAIS)

1-Formalmente Defina:
Classe P- Conjunto de problemas de decisão concretos solucionáveis em tempo polinomial
Classe NP- É a classe de linguagens que podem ser verificadas por um algoritimo de verificação
Classe Np completo- é o conjunto de linguagens L C{0,1}^ que são NP (L E NP) e redutíveis polinomialmente a outra linguagem L' E NP.

2-Como uma linguagens Formais nos ajudam a provar NP-Completude?
As linguagens Formais descrevem problemas concretos que mais facilmente podem ser transcritos em circuitos booleanos que são provados por satisfabilidade como NP completo.

3--O que é um algoritmo de verificação?
Um algoritimo de Redução é...Enquanto um algoritimo de verificação é um algoritimo de dois argumentos , uma string de entrada comum x; e uma string binária y chamada de certificado, e verifica em tempo polinomial se para a astirng x,existe um certificado y.

4- O que é apresentado quando se prova que um problema é NP-Completo ?
É demonstrado uma afirmação sobre o quão difícil é em vez de quão fácil é. Não é tentando provar a existência de um algoritmo eficiente, mas sim que nenhum algoritmo eficiente provavelmente existirá

5-O que é um problema de Decisão?

É um problema onde a resposta desejada é "sim " ou não" (verdadeir/falso) (0,1) com base em uma condição especifica que precisa ser verificada.

6-O que é o problema de satisfação de circuito ?
É um problema que consiste em determinar se existe uma atribuição de valores de entrada que faça com que a saída de um circuito lógico seja verdadeira.

7-O que é um Problema Abstrato?
Um problema abstrato Q é uma relação binária em um conjunto I de instancias de problemas e um conjunto S de soluções de Problemas.

8-O que é um Problema de Decisão Abstrato?
Uma função que mapeia o conjunto de instâncias I par ao conjunto de soluções {0,1}

9-O que é um Problema Concreto ?
Um problema cujo conjunto de instâncias é um conjunto de strings binárias

10-O que é um problema de Otimização?

Um problema de otimização é aquele em que o objetivo é encontrar a melhor solução possível (ótima) dentre um conjunto de soluções viáveis, de acordo com um determinado critério ou função objetivo

11- Identifique a relação existente entre Problemas de Otimização e Problemas de Decisão que pode ajudar na prova de um problema NP-Completo.

Para muitos problemas de otimização,existe uma versão correspondente em formato de decisão,logo provar que a versão de decisão de um problema é NP-Completo implica que a versão de otimização é,no mínimo tão difícil quanto . Dessa forma, a resolução de um problema de decisão pode ser usada para construir a solução para a o problema de otimização equivalente.

12- O que é um algoritmo de redução ?

Um algoritimo de tempo polinomial F que calcula uma função de redução que
É uma função que transforma em tempo polinomial uma instancia de um problema A em um instancia de um problema B, de forma que a solução de B forneça uma solução correspondente para A.

13- Como uma algoritimo de Redução nos ajuda na prova de um problema NPCompleto?

Um algoritmo de redução mostra que um problema é NP-Completo ao transformar, em tempo polinomial, um problema NP-Completo conhecido no problema em questão. Se o novo problema puder ser resolvido eficientemente, então o problema conhecido também pode ser resolvido, evidenciando que o novo problema é pelo menos tão difícil quanto qualquer problema em NP.

14-Liste os passos para provar que um problema pertence a classe de Problemas NPCompleta :

1- Mostar que o problema esta em NP
2- Escolher um problema conhecido NP-completo
3- Construir uma redução polinomial
4- Mostrar que a redução é valida
5- Concluir que o novo problema é tanto pertencente a NP quanto a NP-dificil.

15-Dizemos que intuitivamente um algoritmo A verifica uma linguagem L se ?

Se para qualquer sequencia X pertencente a L existe um certificado y que A pode usar para provar X pertencente a L

16-Intuitivamente dizemos que, um problema Q pode ser reduzido a outro problema Q' se?

Se qualquer instancia de Q puder ser facilmente reformulada como uma instancia de Q' cuja solução fornece uma solução para a instancia de Q.